

Text2Flow: Automating Workflow Creation with AI

Author: wen_gong@vanguard.com

Date: 2025-07-21

Abstract

We present Text2Flow, a novel framework that enables non-technical users to create complex workflows through natural language descriptions. By leveraging Large Language Models (LLMs) and integrating Mermaid diagramming with Prefect workflow orchestration, Text2Flow bridges the gap between business requirements and technical implementation. Our preliminary results demonstrate successful generation of executable Python workflows from simple text descriptions, with visual validation steps that ensure accuracy and user understanding.

1. Motivations

1.1 The Workflow Creation Challenge

Modern organizations increasingly rely on automated workflows to streamline operations, from data processing pipelines to business process automation. However, creating these workflows presents several critical challenges:

- **Technical Barrier:** Traditional workflow creation requires deep programming knowledge and familiarity with specific frameworks like Airflow, Prefect, or Temporal. This creates a bottleneck where business domain experts must translate their requirements to technical teams, often losing crucial context in the process.
- **Communication Gap:** Business stakeholders understand their processes intuitively but struggle to communicate complex logic, conditional branches, and error handling requirements to developers. This leads to multiple iteration cycles and potential misunderstandings.
- **Time-to-Market:** The current workflow development cycle involves requirement gathering, technical specification, development, testing, and deployment. For simple workflows, this process can take weeks when the actual business logic could be expressed in minutes.
- **Maintenance Overhead:** As business requirements evolve, updating workflows requires technical intervention, creating ongoing dependencies and slowing adaptation to changing needs.

1.2 The AI Opportunity

Recent advances in Large Language Models have demonstrated remarkable capabilities in:

- **Code Generation:** Models like GPT-4 and Claude can generate high-quality code from natural language descriptions
- **Reasoning:** Modern LLMs can understand complex logical relationships and conditional flows
- **Domain Understanding:** These models have been trained on vast amounts of business and technical documentation, enabling them to bridge domain-specific knowledge gaps

1.3 Market Impact

The workflow automation market is projected to reach \$52.5 billion by 2028, with low-code/no-code solutions driving significant growth. Text2Flow addresses this opportunity by

- **Democratizing Workflow Creation:** Enabling business users to create workflows without technical expertise
- **Reducing Development Costs:** Eliminating the need for specialized developers for simple workflow tasks
- **Accelerating Innovation:** Allowing rapid prototyping and iteration of business processes

2. Approach

We propose “Text2Flow” - a framework that automates workflow creation through AI-assisted natural language processing. Our approach integrates two proven open-source technologies:

- **Mermaid**[1]: An open-source diagramming tool that generates visual diagrams from text-based descriptions
- **Prefect**[2]: A modern workflow orchestration framework for building resilient data pipelines in Python

2.1 Framework Architecture

Text2Flow operates through a four-step process:

1. **Natural Language Input:** Users describe their desired workflow in plain English, including business logic, conditions, and data flow requirements.
2. **AI-Powered Diagram Generation:** An LLM with reasoning capabilities (GPT-4 or Claude) processes the text description and generates corresponding Mermaid flowchart code, capturing the logical structure and decision points.
3. **Visual Validation:** The Mermaid code is rendered into an interactive visual diagram, allowing users to review and validate the workflow logic before implementation.
4. **Code Generation:** Upon approval, the system generates executable Python code using the Prefect framework, including proper task definitions, error handling, and logging.

2.2 Implementation Details

Our implementation leverages: - **Streamlit** for the user interface, providing an intuitive web-based experience

- **OpenAI/Anthropic APIs** for LLM integration with structured prompting
- **Mermaid.js** for real-time diagram rendering
- **Prefect 2.0** as the target execution environment

The system includes validation mechanisms to ensure generated code follows best practices and includes appropriate error handling, logging, and monitoring capabilities.

3. Preliminary Results

We validated our approach using a representative weather monitoring use case that demonstrates typical workflow patterns: data fetching, processing, conditional logic, and multiple output paths.

3.1 Input: Natural Language Description

Fetch weather data,
process it,
check temperature,
If above 30°C, send heat alert;
else log temperature.
Finally Update weather dashboard

3.2 Generated Mermaid Diagram

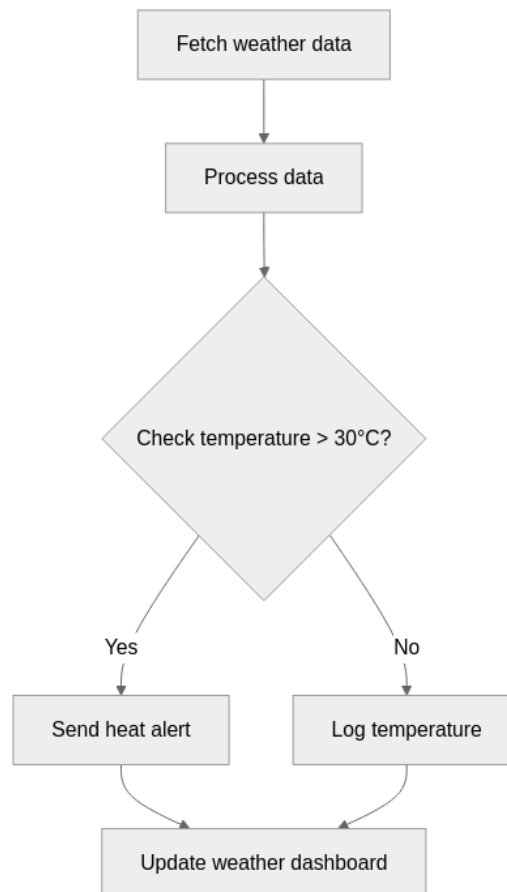


Figure 1: Flowchart

The system successfully generated the following visual workflow representation:

flowchart TD

```
A[Fetch weather data] --> B[Process data]
B --> C{Check temperature > 30°C?}
C -- Yes --> D[Send heat alert]
C -- No --> E[Log temperature]
D --> F[Update weather dashboard]
E --> F
```

3.3 Generated Executable Code

Text2Flow produced fully functional Python code using Prefect decorators and best practices:

```

from prefect import flow, task
from prefect.logging import get_run_logger
import random

@task
def fetch_weather_data() -> float:
    """Fetch weather data and return the current temperature."""
    temperature = random.uniform(25.0, 35.0) # Simulate temperature between 25°C and 35°C
    logger = get_run_logger()
    logger.info(f"Fetch weather data: {temperature:.2f}°C")
    return temperature

@task
def process_data(temperature: float) -> None:
    """Process the fetched temperature data."""
    logger = get_run_logger()
    logger.info(f"Processing temperature data: {temperature:.2f}°C")

@task
def check_temperature(temperature: float) -> bool:
    """Check if the temperature is greater than 30°C."""
    is_hot = temperature > 30.0
    logger = get_run_logger()
    logger.info(f"Temperature check: {'Hot' if is_hot else 'Not Hot'} ({temperature:.2f}°C)")
    return is_hot

@task
def send_heat_alert() -> None:
    """Send a heat alert notification."""
    logger = get_run_logger()
    logger.warning("Heat alert: Temperature exceeds 30°C!")

@task
def log_temperature(temperature: float) -> None:
    """Log the current temperature."""
    logger = get_run_logger()
    logger.info(f"Logging temperature: {temperature:.2f}°C")

@task
def update_weather_dashboard() -> None:
    """Update the weather dashboard with the latest data."""
    logger = get_run_logger()
    logger.info("Weather dashboard updated.")

@flow
def weather_monitoring_workflow() -> None:
    """Main workflow to monitor weather and handle alerts."""
    temperature = fetch_weather_data()
    process_data(temperature)

```

```

    if check_temperature(temperature):
        send_heat_alert()
    else:
        log_temperature(temperature)

    update_weather_dashboard()

if __name__ == "__main__":
    weather_monitoring_workflow()

```

3.4 Execution Results

The generated workflow executed successfully in the Prefect environment, demonstrating: - Proper task orchestration and dependency management - Comprehensive logging and monitoring - Correct conditional logic implementation - Error handling and resilience features

4. Current Capabilities and Limitations

4.1 Strengths

- **High Accuracy:** Both GPT-4 and Claude consistently generate correct Mermaid diagrams for workflows with up to 10 decision points
- **Code Quality:** Generated Python code follows Prefect best practices and includes proper typing and documentation
- **User Experience:** The visual validation step significantly improves user confidence and reduces errors

4.2 Current Limitations

- **Complexity Boundaries:** Performance degrades with workflows containing more than 15 interconnected tasks
- **Domain Specificity:** Currently optimized for data processing workflows; business process workflows require additional prompt engineering
- **Error Recovery:** Limited automatic correction capabilities when initial generation fails

5. Future Research Directions

5.1 Complex Workflow Support

We plan to extend Text2Flow to handle more sophisticated use cases, including:

- **Financial Analysis Automation:** Multi-step data analysis workflows with dynamic branching based on market conditions
- **Business Process Automation:** Cross-system integration workflows with human-in-the-loop approval steps

5.2 Integration with Advanced AI Orchestration

We are exploring integration with **FlowX**[3], an advanced AI agent orchestration platform that provides:

- **Intelligent Agent Coordination:** Enhanced decision-making capabilities for complex workflows

- **Neural Pattern Recognition:** Automatic optimization of workflow patterns based on execution history
- **Sophisticated Memory Systems:** Context-aware workflow adaptation based on previous executions

This integration would enable Text2Flow to generate not just static workflows, but adaptive, learning systems that improve over time.

6. Conclusion

Text2Flow demonstrates the potential for AI-powered workflow automation to democratize complex process creation. By combining natural language processing with proven orchestration frameworks, we enable business users to create sophisticated workflows without technical expertise.

Our preliminary results show promising accuracy and usability, while identifying clear paths for enhancement. The framework’s modular design allows for incremental improvements and integration with emerging AI technologies.

As organizations continue to seek agility and automation, Text2Flow represents a significant step toward truly accessible workflow creation, potentially transforming how businesses approach process automation and digital transformation.

References

- [1] Mermaid - Generation of diagrams from text in a similar manner as markdown. URL: <https://www.wikiwand.com/en/mermaid>
- [2] Prefect - Modern workflow orchestration framework for building resilient data pipelines. URL: <https://github.com/PrefectHQ/prefect>
- [3] FlowX - Advanced AI agent orchestration platform for intelligent workflow coordination. URL: <https://github.com/sethdford/flowx>

Appendix:

Screenshots and Demo Materials

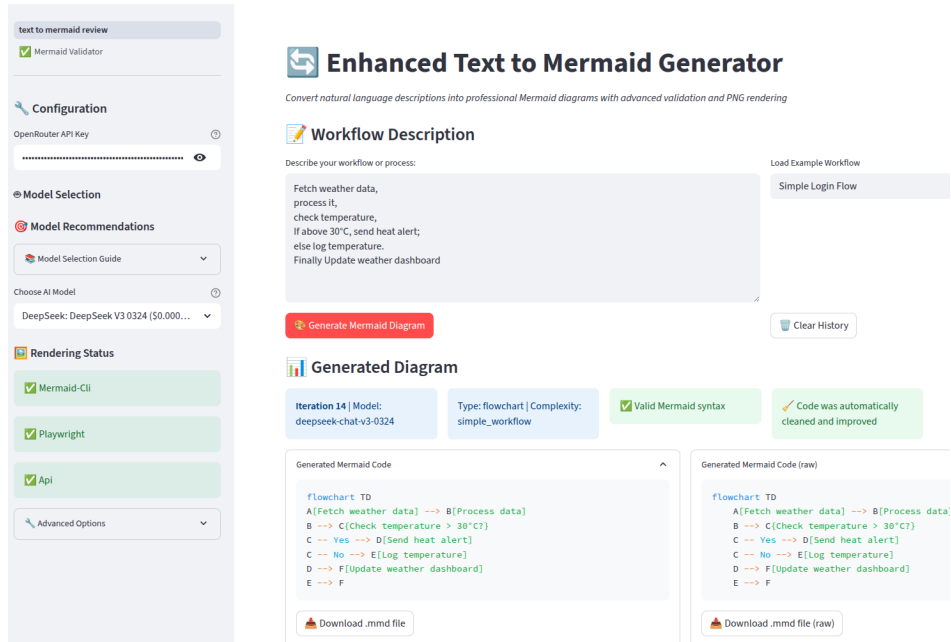


Figure 2: Natural language workflow description interface

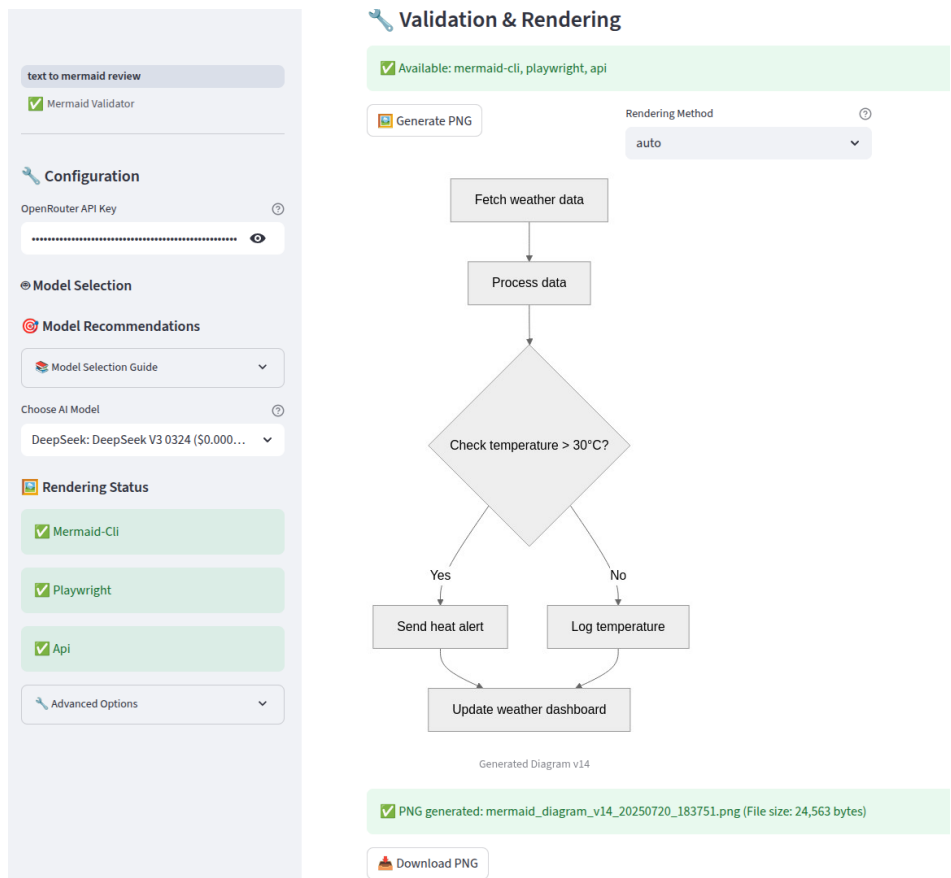


Figure 3: Auto-generated workflow visualization for validation

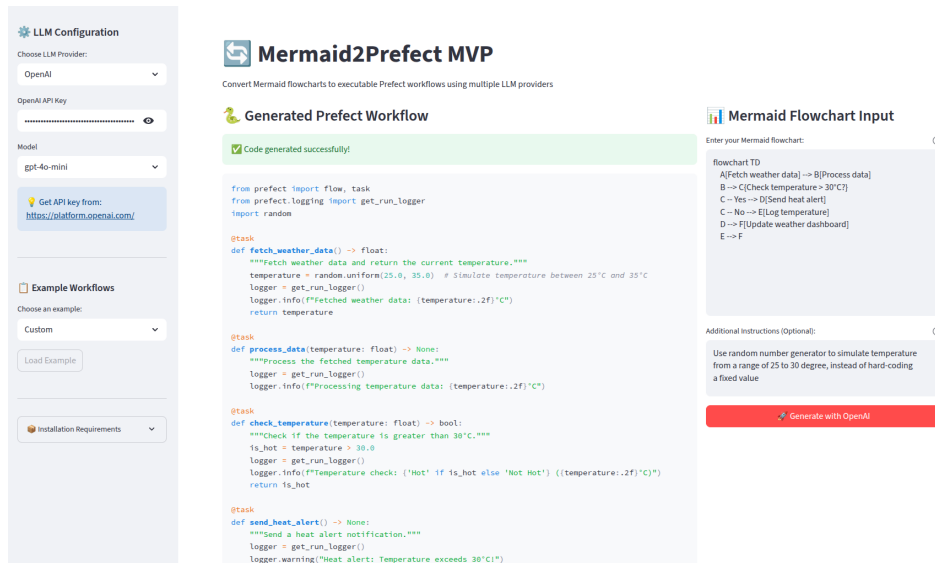


Figure 4: Executable Prefect workflow code

```
(ai_workflow) papagame@papa-game:~/projects/digital-duck/mermaid2prefect/tmp/st_mermaid2prefect/data$ python temp_monitor-gpt-4o-mini-v2.py
18:54:12.014 | INFO | prefect - Starting temporary server on http://127.0.0.1:8095
See https://docs.prefect.io/v3/concepts/server#how-to-guides for more information on running a dedicated Prefect server.
18:54:14.358 | INFO | Flow run 'plouant-shark' - Beginning flow run 'plouant-shark' for flow 'weather-monitoring-workflow'
18:54:14.403 | INFO | Task run 'fetch_weather_data-6fb' - Fetched weather data: 26.38°C
18:54:14.404 | INFO | Task run 'process_data-b6e' - Processing temperature data: 26.38°C
18:54:14.441 | INFO | Task run 'process_data-b6e' - Finished in state Completed()
18:54:14.469 | INFO | Task run 'check_temperature-780' - Temperature check: Not Hot (26.38°C)
18:54:14.471 | INFO | Task run 'check_temperature-780' - Finished in state Completed()
18:54:14.498 | INFO | Task run 'log_temperature-6ca' - Logging temperature: 26.38°C
18:54:14.500 | INFO | Task run 'log_temperature-6ca' - Finished in state Completed()
18:54:14.525 | INFO | Task run 'update_weather_dashboard-eef' - Weather dashboard updated.
18:54:14.527 | INFO | Task run 'update_weather_dashboard-eef' - Finished in state Completed()
18:54:14.547 | INFO | Flow run 'plouant-shark' - Finished in state Completed()
18:54:14.557 | INFO | prefect - Stopping temporary server on http://127.0.0.1:8095
(ai_workflow) papagame@papa-game:~/projects/digital-duck/mermaid2prefect/tmp/st_mermaid2prefect/data$ python temp_monitor-gpt-4o-mini-v2.py
18:54:23.271 | INFO | prefect - Starting temporary server on http://127.0.0.1:8372
See https://docs.prefect.io/v3/concepts/server#how-to-guides for more information on running a dedicated Prefect server.
18:54:25.011 | INFO | Flow run 'jumping-collie' - Beginning flow run 'jumping-collie' for flow 'weather-monitoring-workflow'
18:54:25.655 | INFO | Task run 'fetch_weather_data-d95' - Fetched weather data: 34.83°C
18:54:25.657 | INFO | Task run 'fetch_weather_data-d95' - Finished in state Completed()
18:54:25.693 | INFO | Task run 'process_data-13b' - Processing temperature data: 34.83°C
18:54:25.695 | INFO | Task run 'process_data-13b' - Finished in state Completed()
18:54:25.723 | INFO | Task run 'check_temperature-8a3' - Temperature check: Hot (34.83°C)
18:54:25.725 | INFO | Task run 'check_temperature-8a3' - Finished in state Completed()
18:54:25.753 | WARNING | Task run 'send_heat_alert-e48' - Heat alert: Temperature exceeds 30°C!
18:54:25.755 | INFO | Task run 'send_heat_alert-e48' - Finished in state Completed()
18:54:25.781 | INFO | Task run 'update_weather_dashboard-4e0' - Weather dashboard updated.
18:54:25.783 | INFO | Task run 'update_weather_dashboard-4e0' - Finished in state Completed()
18:54:25.804 | INFO | Flow run 'jumping-collie' - Finished in state Completed()
18:54:25.814 | INFO | prefect - Stopping temporary server on http://127.0.0.1:8372
(ai_workflow) papagame@papa-game:~/projects/digital-duck/mermaid2prefect/tmp/st_mermaid2prefect/data$
```

Figure 5: Prefect execution log